TRƯỜNG ĐẠI HỌC SÀI GÒN (SGU) • KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO PHẦN 1: MÔ HÌNH SẢN PHẨM & ĐA HÌNH BẢO HÀNH

ĐỀ TÀI: QUẢN LÝ CỬA HÀNG THỜI TRANG (JAVA OOP) • PHẦN 1

Phân công: Thành viên 1 | GVHD: Thầy Trần Ngọc Minh Hiếu (tnmhieu@sgu.edu.vn)

HƯỚNG DẪN HỌC & BẢO VỆ VẤN ĐÁP ĐỒ ÁN OOP (ĐIỂM 10/10)

> **Phân công thành viên:** Thành viên 1

> **Các lớp phụ trách (8 lớp & interface):** `SanPhamThoiTrang` , `AoKhoac` , `AoThun` , `QuanJean` , `PhuKien` , `VayDam` , `GiayDep` , `IBaoHanh` , `IGiamGia` .

> **Nhiệm vụ cốt lõi:** Nắm vững cấu trúc cây kế thừa sản phẩm, lớp trừu tượng, phương thức trừu tượng, tính đóng gói dữ liệu và cơ chế đa hình bảo hành qua interface.

1. TỔNG QUAN CÂY PHÂN CẤP SẢN PHẨM THỜI TRANG

```
<<Interface>> IGiamGia
                          ^
                          | implements
               [SanPhamThoiTrang] (abstract)
               /   |     |    \     \      \
              /    |     |     \     \      \
     [AoKhoac] [AoThun] [QuanJean] [PhuKien] [VayDam] [GiayDep]
         |                             |                  |
         +------------ implements ----+------------------+
                                       v
                            <<Interface>> IBaoHanh
```

- **Mối quan hệ IS-A (Là một):** Áo khoác, Áo thun, Quần jean, Phụ kiện, Váy đầm, Giày dép đều là một Sản phẩm thời trang (`extends SanPhamThoiTrang`).
- **Mối quan hệ CAN-DO (Có khả năng):** Áo khoác, Phụ kiện, Giày dép có thể bảo hành (`implements IBaoHanh`). Quần áo thông thường không có bảo hành.

2. GIẢI THÍCH CHI TIẾT TỪNG ĐOẠN CODE CỐT LÕI

Đoạn 1: Khai báo lớp trừu tượng `SanPhamThoiTrang`

```
public abstract class SanPhamThoiTrang implements IGiamGia {
    public static final double TI_LE_VAT = 0.10;
    private static int tongSoLuongSanPhamDaTao = 0;

    private String Type;
    private String MaSP;
    private String TenSP;
    private double GiaSP;
```

- **Giải thích từng dòng code:**

- `public abstract class SanPhamThoiTrang` : Khai báo lớp trừu tượng. Lớp này đóng vai trò lớp cha định hình khuôn mẫu cho mọi sản phẩm, **không thể khởi tạo trực tiếp** bằng lệnh `new SanPhamThoiTrang()` mà chỉ dùng làm tham chiếu đa hình.
- `implements IGiamGia` : Cam kết hiện thực interface giảm giá sản phẩm.
- `public static final double TI_LE_VAT = 0.10` : Thuộc tính static hằng số (10% thuế VAT). Dùng chung cho toàn bộ sản phẩm trong cửa hàng, không tốn bộ nhớ cấp phát riêng cho từng đối tượng.
- `private static int tongSoLuongSanPhamDaTao = 0` : Biến static đếm tổng số sản phẩm đã được tạo trong bộ nhớ.
- `private String Type, MaSP, TenSP; private double GiaSP;` : Tính **Đóng gói (Encapsulation)**. Toàn bộ thuộc tính cơ sở được đặt `private` để bảo vệ dữ liệu, chống việc gán giá trị bừa bãi từ bên ngoài.

Đoạn 2: Hàm khởi tạo (Constructor) có tham số & Ràng buộc dữ liệu

```
public SanPhamThoiTrang(String Type, String MaSP, String TenSP, double GiaSP) {
        this.Type = (Type != null) ? Type : "ChuaPhanLoai";
        this.MaSP = (MaSP != null) ? MaSP : "";
        this.TenSP = (TenSP != null) ? TenSP : "";
        this.GiaSP = (GiaSP >= 0) ? GiaSP : 0;
        tongSoLuongSanPhamDaTao++;
    }
```

- **Giải thích từng dòng code:**
- `this.Type = (Type != null) ? Type : "ChuaPhanLoai"` : Kiểm tra null an toàn. Tránh lỗi `NullPointerException` khi thao tác chuỗi.
- `this.GiaSP = (GiaSP >= 0) ? GiaSP : 0` : Ràng buộc giá tiền không được âm. Nếu truyền số âm sẽ tự động đưa về 0.
- `tongSoLuongSanPhamDaTao++` : Mỗi khi từ khóa `new` được gọi ở bất kỳ lớp con nào (`AoKhoac` , `AoThun` ...), biến static này tự động tăng 1 để quản lý số lượng đối tượng đã sinh ra.

Đoạn 3: Các phương thức trừu tượng (Abstract Methods)

```
public abstract String getStringSaveFile();
    public abstract String getThongTinRieng();
```

- **Giải thích từng dòng code:**
- Phương thức trừu tượng chỉ có phần khai báo, **không có thân hàm `{}`**, kết thúc bằng dấu chấm phẩy `;` .
- Ép buộc 100% các lớp con kế thừa (`AoKhoac` , `AoThun` , `QuanJean` ...) **bắt buộc phải ghi đè (@Override)** để định nghĩa cách lấy thông tin riêng và chuỗi lưu file phù hợp với đặc thù từng loại mặt hàng.

Đoạn 4: Tính toán giá sau giảm & Thuế VAT

```
public double VATcalculation() {
        return this.GiaSP * TI_LE_VAT;
    }

    @Override
    public double calculateAfterDiscount(double discountPercentage) {
```

```
        return this.GiaSP - (this.GiaSP * discountPercentage / 100);
    }
```

- **Giải thích từng dòng code:**
- `VATcalculation()` : Tính tiền thuế GTGT bằng cách lấy đơn giá nhân với hằng số `TI_LE_VAT` (10%).
- `calculateAfterDiscount(...)` : Hiện thực từ Interface `IGiamGia` . Công thức: `Giá sau giảm = Giá gốc - (Giá gốc * % giảm / 100)` . Cung cấp khả năng tính giá linh hoạt cho các đợt khuyến mãi.

Đoạn 5: Setter bảo vệ dữ liệu & Chống lỗi định dạng file CSV

```
public void setTenSP(String TenSP) {
        if (TenSP != null && !TenSP.trim().isEmpty()) {
            this.TenSP = TenSP.trim().replace(",", " ");
        }
    }

    public boolean setGiaBan(double GiaBanMoi) {
        if (GiaBanMoi >= 0) {
            this.GiaSP = GiaBanMoi;
            return true;
        } else {
            System.out.println("Gia ban phai lon hon hoac bang 0!");
            return false;
        }
    }
```

- **Giải thích từng dòng code:**
- `replace(",", " ")` : Do hệ thống lưu file `products.txt` dạng dòng phân tách bằng dấu phẩy (CSV format), nếu tên sản phẩm chứa dấu phẩy (ví dụ: "Áo gió, nam, cao cấp") thì khi đọc file bằng hàm `split(",")` sẽ làm sai số lượng cột. Thao tác này khử dấu phẩy triệt để để bảo vệ tính toàn vẹn của file dữ liệu.
- `setGiaBan` : Trả về `boolean` báo hiệu thành công hay thất bại. Chặn không cho cập nhật giá âm.

Đoạn 6: Lớp con `AoKhoac` kế thừa & Hiện thực `IBaoHanh`

```
public class AoKhoac extends SanPhamThoiTrang implements IBaoHanh {
    private boolean hasHat;

    public AoKhoac(String Type, String MaSP, String TenSP, double GiaSP, boolean hasHat) {
        super(Type, MaSP, TenSP, GiaSP);
        this.hasHat = hasHat;
    }

    @Override
    public String getThongTinRieng() {
        return "Co non: " + (this.hasHat ? "Co" : "Khong");
    }

    @Override
    public void printWarrantyCard() {
        System.out.println("+------------------------------------------------------------+");
        System.out.println("|                    PHIEU BAO HANH                          |");
        System.out.println("+------------------------------------------------------------+");
        System.out.printf("| Ten san pham      : %-33s |\n", getTenSP());
        System.out.printf("| Ma san pham       : %-33s |\n", getMaSP());
```

```
        System.out.printf("| Thoi han bao hanh : %-33s |\n", getWarrantyPeriod() + " thang");
        System.out.println("+-------------------------------------------------------+");
    }

    @Override
    public int getWarrantyPeriod() {
        return 12;
    }

    @Override
    public String getStringSaveFile() {
        return getType() + "," + getMaSP() + "," + getTenSP() + "," + getGiaBan() + "," +
this.hasHat;
    }
}
```

- **Giải thích từng dòng code:**
- `extends SanPhamThoiTrang` : Kế thừa toàn bộ mã nguồn quản lý mã, tên, giá, VAT từ lớp cha.
- `implements IBaoHanh` : Khẳng định Áo khoác có chế độ bảo hành chính hãng.
- `super(Type, MaSP, TenSP, GiaSP)` : Lời gọi constructor của lớp cha để khởi tạo các thuộc tính chung. Phải nằm ở **dòng đầu tiên** của constructor lớp con.
- `this.hasHat = hasHat` : Khởi tạo thuộc tính riêng của áo khoác (có mũ nón hay không).
- `@Override getThongTinRieng()` : Hiện thực phương thức trừu tượng, trả về chuỗi thông tin riêng hiển thị trên bảng Console.
- `@Override getWarrantyPeriod()` : Trả về thời hạn bảo hành cố định là **12 tháng**.
- `@Override printWarrantyCard()` : Đóng khung in phiếu bảo hành chuẩn format.
- `@Override getStringSaveFile()` : Xuất định dạng 5 cột chuẩn để ghi xuống `products.txt` .

Đoạn 7: Các lớp con chuyên biệt khác

1. **`AoThun`**:
- Thuộc tính riêng: `private String size;`
- Logic chuẩn hóa: Khi khởi tạo hoặc gọi `setSize(String sz)` , tự động chuyển thành chữ hoa `S, M, L, XL` . Nếu nhập sai mặc định gán là `"M"` .
2. **`QuanJean`**:
- Thuộc tính riêng: `private String Style;` (Kiểu dáng: Skinny, Baggy, Ống rộng). Có xử lý lọc bỏ dấu phẩy.
3. **`PhuKien`**:
- Thuộc tính riêng: `private String Material;` (Chất liệu: Da bò, Kim loại...).
- Hiện thực `IBaoHanh` với thời hạn bảo hành **6 tháng**.
4. **`VayDam`**:
- Thuộc tính riêng: `private String chatLieuVai;` (Voan, Lụa, Cotton...).
5. **`GiayDep`**:
- Thuộc tính riêng: `private int sizeGiay;`
- Ràng buộc dữ liệu: Size giày nằm trong khoảng hợp lệ từ **35 đến 45**.
- Hiện thực `IBaoHanh` với thời hạn bảo hành **3 tháng**.

3. BỘ CÂU HỎI VẤN ĐÁP CỦA THẦY HIẾU & CÁCH TRẢ LỜI ĐẠT 10 ĐIỂM

CÂU HỎI VẤN ĐÁP: Tại sao `SanPhamThoiTrang` lại là Abstract Class mà không phải Interface?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, `SanPhamThoiTrang` chứa các thuộc tính trạng thái cụ thể (`Type`, `MaSP`, `TenSP`, `GiaSP`) và các phương thức đã có thân hàm cài đặt sẵn (`VATcalculation`, `formatTien`, `setGiaBan`). Interface không thể lưu trạng thái đối tượng như vậy. Ngoài ra, quan hệ giữa Áo khoác/Áo thun với Sản phẩm là quan hệ bản chất kế thừa **IS-A** (Áo khoác là một sản phẩm), do đó thiết kế Lớp trừu tượng là hoàn toàn chuẩn xác theo nguyên lý OOP.

CÂU HỎI VẤN ĐÁP: Từ khóa `super` trong constructor lớp con dùng để làm gì? Bỏ đi có được không?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, từ khóa `super(...)` dùng để gọi hàm thiết lập của lớp cha nhằm khởi tạo các thuộc tính `Type`, `MaSP`, `TenSP`, `GiaSP` do lớp cha quản lý. Lệnh này bắt buộc phải nằm ở dòng đầu tiên trong constructor của lớp con. Nếu không ghi tường minh `super(...)` có tham số, Java sẽ tự động gọi `super()` mặc định không tham số của lớp cha.

CÂU HỎI VẤN ĐÁP: Kỹ thuật Đa hình (Polymorphism) thể hiện như thế nào trong phần quản lý sản phẩm này?

TRẢ LỜI MẪU (10 ĐIỂM): Dạ thưa Thầy, đa hình thể hiện rõ rệt ở 2 điểm:

1. **Đa hình qua lớp trừu tượng:** Khi một biến tham chiếu kiểu cha `SanPhamThoiTrang sp` duyệt qua danh sách, gọi `sp.getThongTinRieng()` hay `sp.getStringSaveFile()`, chương trình sẽ tự động kích hoạt phiên bản phương thức tương ứng của từng đối tượng thực tế tại runtime (`AoKhoac`, `AoThun`, `GiayDep`...).
2. **Đa hình qua Interface `IBaoHanh`:** Trong lớp hóa đơn, chương trình kiểm tra `if (sp instanceof IBaoHanh) { ((IBaoHanh) sp).printWarrantyCard(); }`. Dù cùng là giao diện bảo hành, Áo khoác sẽ in 12 tháng, Phụ kiện in 6 tháng và Giày dép in 3 tháng.